Software Architecture and Design Specification

Smart Complaint Management System (SCMS)

Project: Smart Complaint Management System (SCMS)

Version: 1.0

Authors: Team 8

Date: 04-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team 8	Initial SAD derived from SRS v2.0 and STP v1.0: architecture, component and sequence diagrams, API design, security architecture, and requirement traceability.

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Smart Complaint Management System (SCMS). It translates the requirements in the SCMS Software Requirements Specification (SRS) Version 2.0 into a structural description: the components, their responsibilities, the pattern that organizes them, and the key runtime interactions and interfaces. It also records the security architecture and shows how each requirement is traced to a component and a test case.

1.2 Scope

SCMS is a digital civic platform for public grievance lodging, ticket triage, departmental assignment, investigation tracking, and citizen resolution feedback. This document covers the user-facing services: registration and role-based authentication, complaint lodging with multimedia evidence upload, lifecycle tracking and audit trail, admin triage and departmental assignment, resolution verification, automated SMS/email notifications, citizen feedback and rating, and reporting/analytics export.

Out of scope, consistent with the SRS: automated physical on-site maintenance dispatch hardware, internal municipal payroll software, and off-platform legal litigation workflows.

1.3 Audience

Software developers (backend and frontend), QA engineers, security auditors, system administrators and department officers, academic course evaluators, and future maintenance teams.

1.4 Definitions

Term	Meaning
SCMS	Smart Complaint Management System
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix
RBAC	Role-Based Access Control (Citizen, Department Staff, Administrator)
SLA	Service Level Agreement: maximum stipulated time to acknowledge or resolve a grievance
JWT	JSON Web Token, used for stateless authentication
MIME / Magic byte	Content-type identifiers; magic bytes are the leading bytes of a file used to verify its real type
ORM	Object-Relational Mapping layer (Django ORM, SQLAlchemy, or Laravel Eloquent)
ACID	Atomicity, Consistency, Isolation, Durability: transactional guarantees of the relational database
ADR	Architecture Decision Record
STRIDE	Threat model categories: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security (TLS 1.3, RFC 8446)
PII	Personally identifiable information

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (goals, stakeholders, component diagram, component descriptions, pattern choice, technology, risks, traceability, and security). Section 4 describes the detailed design (sequence diagrams, API design, error handling, UX, and open issues). Developers should read Sections 3.3, 3.4 and 4.2 first; QA engineers should start with Section 3.8; security reviewers should read Section 3.9.

2.2 Related Documents

- SCMS Software Requirements Specification (SRS), Version 2.0, dated 03-09-2026
- SCMS Software Test Plan (STP), Version 1.0, dated 17-09-2026
- Requirements Traceability Matrix (RTM), SRS Section 8

3. Architecture

3.1 Goals & Constraints

Goals:

- Secure: protect citizen PII, enforce RBAC, TLS 1.3 everywhere, tamper-evident audit trail.
- Reliable: 99.5% monthly availability, daily encrypted backups with restore in under 30 minutes, and graceful degradation when third-party gateways fail.
- Responsive: 90th-percentile response of 2.0 seconds or less at 200 concurrent users; capacity for 500 concurrent authenticated sessions.
- Accessible and usable: WCAG 2.1 Level AA, mobile-first, a complaint can be lodged in a 3-step wizard.
- Modular and testable: service-layer modules enabling 80% or higher unit test coverage.

Constraints:

- Regulatory compliance with national data protection and privacy laws for PII.
- Attachment safeguards: at most 5 files per complaint, 5 MB per file, whitelist of image/jpeg, image/png, and application/pdf.
- Bandwidth economy: responsive over constrained 3G/4G mobile connections.
- Technology: Python 3.11+ (Django 5.x / Flask 3.x) or PHP 8.2+ (Laravel 11.x); PostgreSQL 15+ or MySQL 8.0; Ubuntu 22.04/24.04 LTS; minimum 4 vCPUs and 8 GB RAM.
- Out of scope: physical maintenance dispatch hardware, municipal payroll integration, and off-platform legal workflows.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Citizens (Complainants)	Easy, accessible complaint lodging; privacy of personal data; transparent status tracking; timely notifications; ability to give feedback.
Department Staff Officers	Clear assignments, ability to update milestones and upload resolution proof, reliable notifications.
System Administrators / Municipal Head	Fast triage over large datasets, SLA enforcement, user management, analytics export, auditability.
Security / Compliance	RBAC enforcement, input sanitization, secure credential storage, immutable audit trail, PII protection.

Stakeholder	Concerns
Developers	Modularity, clear component boundaries, testability.
QA Engineers	Requirement-to-component-to-test traceability; mockable external services.
Course Evaluators	Architectural completeness against SE curriculum benchmarks.

3.3 Component (UML) Diagram

Figure 3.1 shows the SCMS components organized into four layers: Client, Presentation, Application Backend, and Data Stores & External Services. Red components are security services, blue are business-logic modules, green are supporting services, orange are data stores, and grey are external services. Dashed arrows are dependencies; only the key links are drawn, and all modules persist data through the ORM layer.

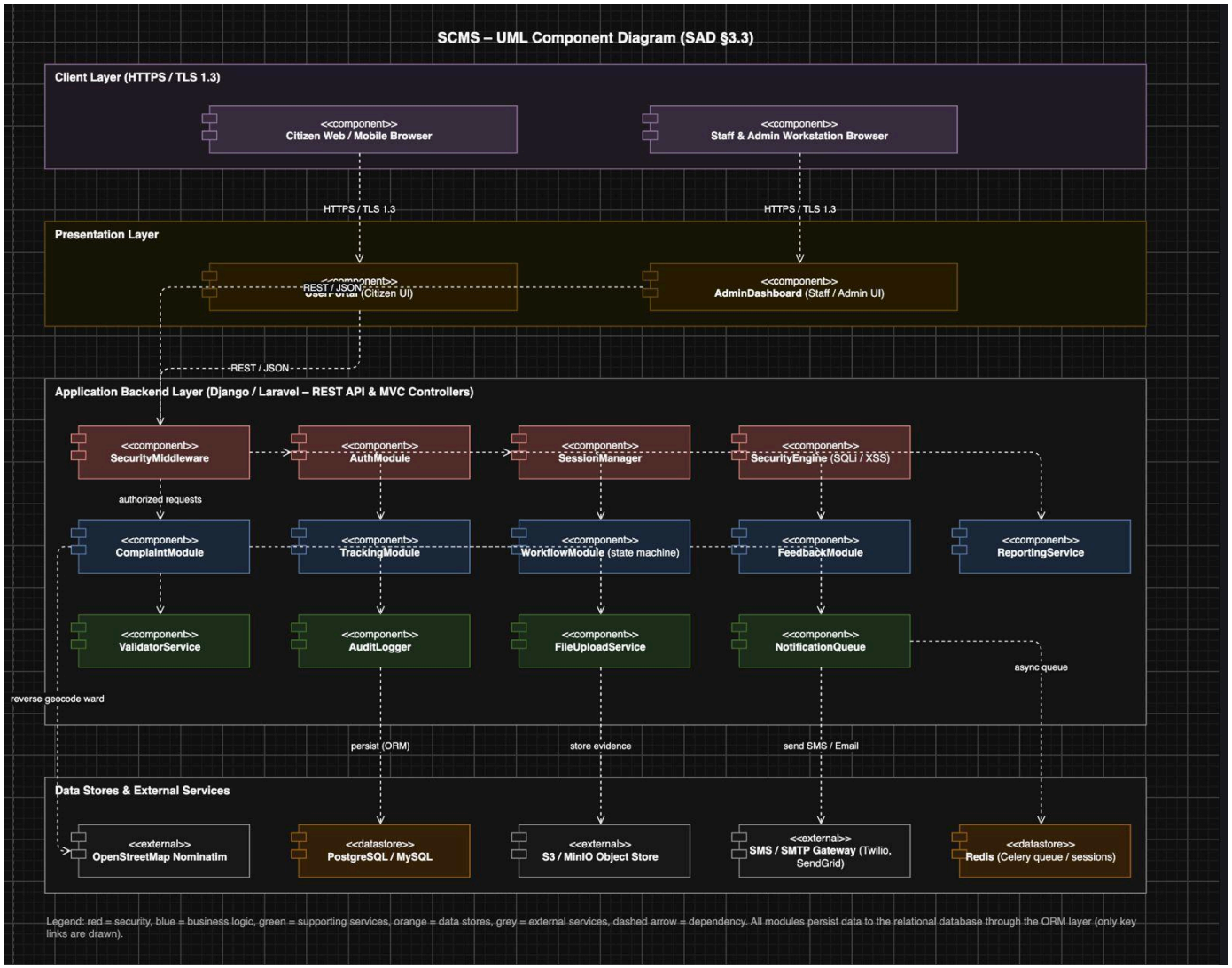


Figure 3.1 – SCMS UML component diagram

3.4 Component Descriptions

Component	Layer	Responsibility
UserPortal	Presentation	Citizen-facing UI: 3-step complaint wizard, tracking search bar, personal dashboard (SCMS-F-010).
AdminDashboard	Presentation	Staff and administrator workspace: sortable tables, triage filters, kanban lifecycle board, resolution modals (SCMS-F-013).
SecurityMiddleware	Application – security	Single entry guard for REST requests: verifies the JWT and enforces RBAC; returns HTTP 403 on cross-role access (SCMS-F-003).
AuthModule	Application – security	Registration, credential authentication, salted Argon2id/bcrypt hashing, and 30-minute lockout after 5 failed attempts (SCMS-F-001, F-002, F-004, SR-002).
SessionManager	Application – security	Issues and rotates session tokens and invalidates sessions after 30 minutes idle (SCMS-SR-004).
SecurityEngine	Application – security	Sanitizes input to prevent SQL injection and stored/reflected XSS (SCMS-SR-003).
ComplaintModule	Application – business	Creates complaints with status Submitted, generates the immutable tracking ID, and accepts citizen note appends while a case is Under Review (SCMS-F-005, F-007, F-012).
TrackingModule	Application – business	Public tracking search by ID with PII masking (SCMS-F-009).
WorkflowModule	Application – business	Grievance state machine: assignment, resolution verification (note of at least 20 characters plus proof photo), and rejection with justification code (SCMS-F-014, F-015, F-016).
FeedbackModule	Application – business	Star rating (1 to 5) and feedback within 7 days of closure; updates the officer performance index (SCMS-F-018).
ReportingService	Application – business	Streams filtered CSV/PDF exports within 3 seconds for up to 10,000 rows (SCMS-F-019).
ValidatorService	Application – supporting	Client-side and server-side validation of mandatory fields and payloads (SCMS-F-008).
AuditLogger	Application – supporting	Writes immutable audit entries: previous status, new status, actor ID, and timestamp (SCMS-F-011).
FileUploadService	Application – supporting	Enforces file count and size limits, MIME and magic-byte checks, and stores evidence in S3-compatible storage (SCMS-F-006, SR-005).
NotificationQueue	Application – supporting	Asynchronous queue (Celery/Redis) dispatching SMS and email within 30 seconds of the status commit, with a dead-letter retry queue (SCMS-F-017).
PostgreSQL / MySQL	Data store	Relational schema with ACID transactions and the audit event tables, accessed through the ORM with parameterized queries.
Redis	Data store	Celery broker for the notification queue; optionally server-side encrypted sessions.
S3 / MinIO Object Store	External	Encrypted storage for evidence files over TLS.
SMS / SMTP Gateway	External	Transactional SMS (for example Twilio) and email (for example SendGrid or Amazon SES).
OpenStreetMap Nominatim	External	Reverse geocoding of latitude/longitude into municipal ward addresses.

3.5 Chosen Architecture Pattern and Rationale

SCMS uses a layered, multi-tier architecture (client, presentation, application backend, data and external services) deployed as a single modular backend (a modular monolith) exposing a REST API.

Rationale:

- Clear separation of concerns: security, business logic, and supporting services are separate modules, which supports the 80% unit test coverage goal and independent defect fixing by backend and frontend developers.
- A central guard (SecurityMiddleware) in front of all business modules makes RBAC and token validation a single enforcement point.
- Transactional consistency: complaint state changes and audit entries rely on ACID guarantees of one relational database, which a distributed design would complicate.
- Asynchronous work (notifications) is isolated behind a queue so that gateway failures never abort a complaint commit.

Alternative rejected: Microservices were rejected as overly complex for the expected scale (500 concurrent sessions, one municipal deployment) and for a student team; they would add distributed transactions, deployment overhead, and more cross-service security surface.

Key architecture decisions (ADR summary)

ID	Decision	Rationale
ADR-01	Layered modular monolith with REST API	Simplicity, ACID consistency, easier testing and deployment at project scale.
ADR-02	Notifications through an asynchronous Celery/Redis queue with dead-letter retry	Meets the 30-second dispatch target and keeps gateway outages from blocking commits (SRS Section 6).
ADR-03	S3-compatible object storage for evidence	Avoids storage exhaustion on the application server; encrypted storage over TLS.
ADR-04	Central RBAC guard plus separate SecurityEngine for input sanitization	One enforcement point for authorization; consistent sanitization across all endpoints.
ADR-05	Audit data in an append-only (immutable) table written by AuditLogger	Accountability and tamper evidence for all state changes.

3.6 Technology Stack & Data Stores

Area	Technology (per SRS)
Backend	Python 3.11+ with Django 5.x or Flask 3.x, or PHP 8.2+ with Laravel 11.x (final choice open; see Section 4.6)
Database	PostgreSQL 15+ or MySQL 8.0 Enterprise with ACID transactions, accessed via ORM (Django ORM, SQLAlchemy, or Laravel Eloquent)
Async processing	Celery with Redis broker
Object storage	AWS S3 or MinIO (S3-compatible API over TLS)
Notification gateways	Twilio (SMS); SendGrid or Amazon SES (SMTP/TLS)
Geocoding	OpenStreetMap Nominatim API

Area	Technology (per SRS)
Transport and sessions	HTTP/2 over TLS 1.3 only (port 80 redirects to 443); JWT in HttpOnly, Secure, SameSite=Strict cookies, or server-side encrypted Redis sessions; UTF-8 application/json payloads
Client	Chrome 110+, Firefox 115+, Safari 16+, Edge 110+; mobile-first responsive UI
Hosting	Ubuntu 22.04/24.04 LTS on cloud VPS (AWS EC2, DigitalOcean) or municipal on-premises VM; at least 4 vCPUs, 8 GB RAM, SSD

3.7 Risks & Mitigations

Risk	Mitigation
SMS/email gateway outage	Notifications are queued asynchronously; unsent alerts move to a dead-letter retry queue; complaint commits are never aborted. Local mocks (MailHog) are used in test.
Delay in staging build delivery	Request early smoke-test builds from the development sub-team.
Late-discovered critical security findings	Run OWASP ZAP and Burp Suite scans in Phase 3 of testing, not only just before submission.
Insufficient infrastructure for 500-session load tests	Cloud VM with distributed JMeter load generators.
Slow filtering on large datasets (100,000 records)	Database indexing on status and ward columns; filter target of 1 second.
Storage exhaustion or malicious uploads	File count and size limits, whitelist of types, magic-byte inspection, and external object storage.
Data loss or extended outage	Automated daily encrypted backups retained 365 days; restore to staging in under 30 minutes (TC-OPS-02).
Third-party sandbox downtime in test (Twilio, SendGrid, MinIO, Nominatim)	Fall back to local mocks (MailHog, MinIO, local Nominatim).
Team availability near academic deadlines	Buffer time built into Phase 4 (UAT and regression).

3.8 Traceability to Requirements

Every SRS requirement maps to a target component (from the RTM, SRS Section 8) and to a test case in the Software Test Plan. Where the RTM names a module that is not drawn as its own component (for example WebCore, Infrastructure, Web Server, Frontend UI, Storage Infra), the responsible element is shown in the Component column.

Req ID	Requirement	Component	Test Case	Section
SCMS-F-001	Citizen Registration	AuthModule	TC-AUTH-01	SRS 4.1
SCMS-F-002	Secure Authentication	AuthModule	TC-AUTH-02	SRS 4.1
SCMS-F-003	Role-Based Access Control	SecurityMiddleware	TC-AUTH-03	SRS 4.1
SCMS-F-004	Account Lockout Protection	AuthModule	TC-AUTH-04	SRS 4.1
SCMS-F-005	Lodge Grievance Form	ComplaintModule	TC-LODGE-01	SRS 4.2
SCMS-F-006	File Evidence Upload	FileUploadService	TC-LODGE-02	SRS 4.2
SCMS-F-007	Unique Tracking ID Generation	ComplaintModule	TC-LODGE-03	SRS 4.2

Req ID	Requirement	Component	Test Case	Section
SCMS-F-008	Payload Field Validation	ValidatorService	TC-LODGE-04	SRS 4.2
SCMS-F-009	Public Tracking Search	TrackingModule	TC-TRACK-01	SRS 4.3
SCMS-F-010	Citizen Dashboard	UserPortal	TC-TRACK-02	SRS 4.3
SCMS-F-011	Audit Status History	AuditLogger	TC-TRACK-03	SRS 4.3
SCMS-F-012	Citizen Append Notes	ComplaintModule	TC-TRACK-04	SRS 4.3
SCMS-F-013	Admin Triage Filtering	AdminDashboard	TC-ADMIN-01	SRS 4.4
SCMS-F-014	Department Assignment	WorkflowModule	TC-ADMIN-02	SRS 4.4
SCMS-F-015	Resolution Verification	WorkflowModule	TC-ADMIN-03	SRS 4.4
SCMS-F-016	Rejection Justification	WorkflowModule	TC-ADMIN-04	SRS 4.4
SCMS-F-017	Automated Notifications	NotificationQueue	TC-NOTIF-01	SRS 4.5
SCMS-F-018	Citizen Star Rating	FeedbackModule	TC-NOTIF-02	SRS 4.5
SCMS-F-019	Analytics Data Export	ReportingService	TC-REP-01	SRS 4.5
SCMS-NF-001	Transaction Response ≤ 2.0 s	Application layer / Database (WebCore)	TC-PERF-01	SRS 5
SCMS-NF-002	99.5% Service Uptime	Infrastructure	TC-OPS-01	SRS 5
SCMS-NF-003	500 Concurrent Session Capacity	Infrastructure	TC-SCALE-01	SRS 5
SCMS-NF-004	WCAG 2.1 AA Compliance	UserPortal / AdminDashboard (Frontend UI)	TC-UX-01	SRS 5
SCMS-NF-005	Daily Backup Retention	Database / Storage infrastructure	TC-OPS-02	SRS 5
SCMS-SR-001	TLS 1.3 Transport Security	Web server	TC-SEC-01	SRS 5.1.2
SCMS-SR-002	Argon2id / bcrypt Hashing	AuthModule	TC-SEC-02	SRS 5.1.2
SCMS-SR-003	Input Sanitization (SQLi / XSS)	SecurityEngine	TC-SEC-03	SRS 5.1.2
SCMS-SR-004	30-Minute Session Timeout	SessionManager	TC-SEC-04	SRS 5.1.2
SCMS-SR-005	Magic-Byte File Whitelisting	FileUploadService	TC-SEC-05	SRS 5.1.2

3.9 Security Architecture

Security objectives from the SRS are confidentiality of citizen PII, integrity of complaint records and timestamps, availability against brute-force and denial-of-service, and accountability through a complete audit trail. The table applies STRIDE threat modeling to the SCMS architecture.

Threat (STRIDE)	SCMS control	Component / Requirement
Spoofing	Salted Argon2id/bcrypt password hashing (cost factor 12 or higher), generic login errors, JWT validation on every request, and a 30-minute lockout after 5 failed attempts.	AuthModule, SecurityMiddleware; F-002, F-004, SR-002
Tampering	TLS 1.3 in transit; parameterized ORM queries; input sanitization; magic-byte inspection to block disguised executables; immutable audit table.	SecurityEngine, FileUploadService, AuditLogger; SR-001, SR-003, SR-005, F-011
Repudiation	Every state change is logged with previous status, new status, actor ID, and timestamp, mapped to an authenticated identity.	AuditLogger; F-011, F-016

Threat (STRIDE)	SCMS control	Component / Requirement
Information disclosure	TLS 1.3 with HTTP redirect to HTTPS, HttpOnly/Secure/SameSite=Strict cookies, masking of PII in public tracking, encrypted evidence storage, no sensitive data in logs.	TrackingModule, S3/MinIO; SR-001, F-009
Denial of service	Account lockout, per-file (5 MB) and per-complaint (5 files) limits, asynchronous queue isolation of gateway work. Request rate limiting on public endpoints is recommended as a further control.	FileUploadService, NotificationQueue; F-004, F-006
Elevation of privilege	Central RBAC guard returning HTTP 403 on cross-role access; token rotation on privilege elevation; 30-minute idle session invalidation.	SecurityMiddleware, SessionManager; F-003, SR-004

Security verification is performed by the TS-SEC suite in the Software Test Plan: Qualys SSL Labs (Grade A), direct database inspection of password hashes, OWASP ZAP and Burp Suite scans, session-timeout and magic-byte tests.

4. Design

4.1 Design Overview

SCMS is designed as layered modules for separation of concerns and easier maintenance. Every request from a browser enters through the presentation layer, is authenticated and authorized by SecurityMiddleware, is sanitized and validated, and is then handled by a business module. State changes are committed in a single database transaction, written to the audit log, and followed by asynchronous notifications. The complaint lifecycle is Submitted, Under Review, Assigned, In Progress, Resolved, Rejected, or Appealed, and states change only through designated workflow events in WorkflowModule.

The use-case diagrams from SRS Section 7 give the functional context for the design that follows.

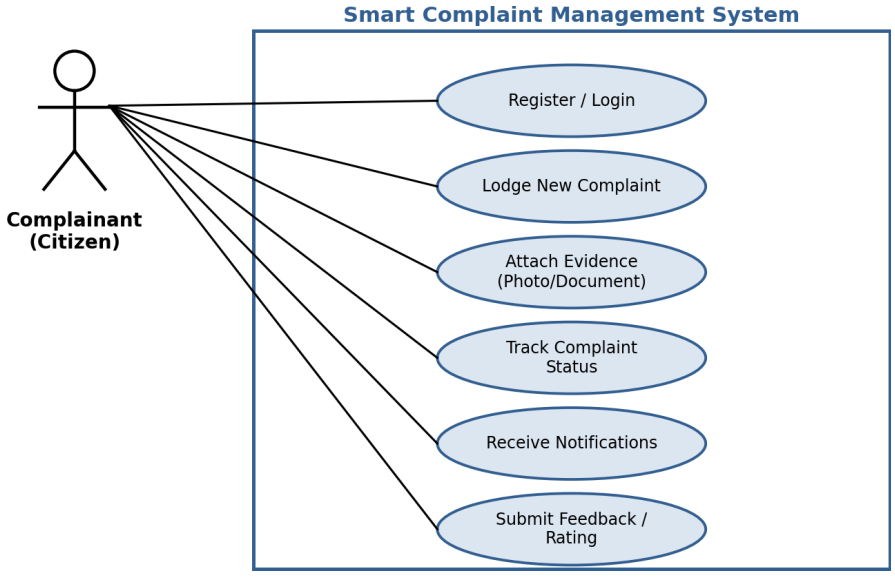


Figure 4.1 – Citizen use-case diagram (SRS Figure 7.1)

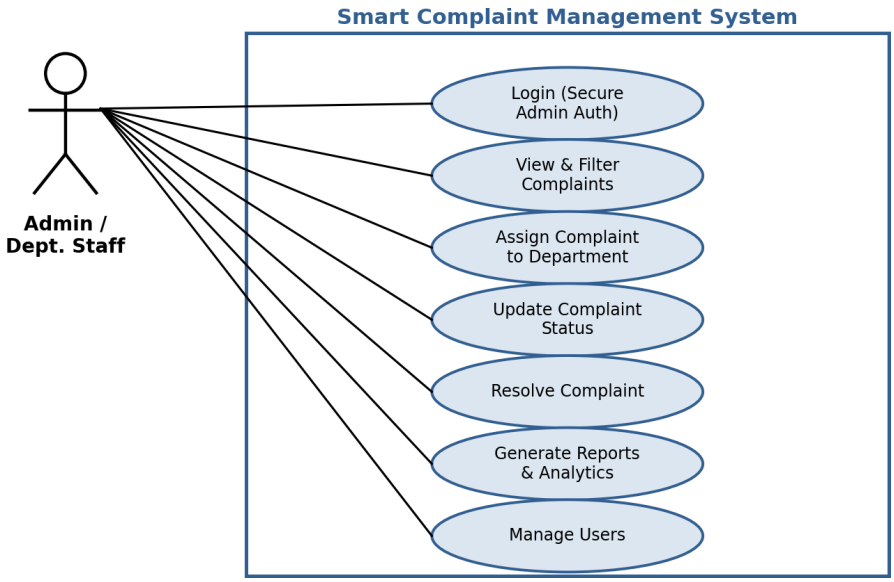


Figure 4.2 – Staff & Administrator use-case diagram (SRS Figure 7.2)

4.2 UML Sequence Diagrams

Sequence Diagram 1 covers the citizen flow for lodging a complaint with evidence (SCMS-F-005 to F-008, SR-003, SR-005). The request is authorized, validated and sanitized, evidence is checked by magic bytes and stored in S3/MinIO, the complaint is created with status Submitted and a tracking ID, the audit entry is written, and an alert is queued asynchronously.

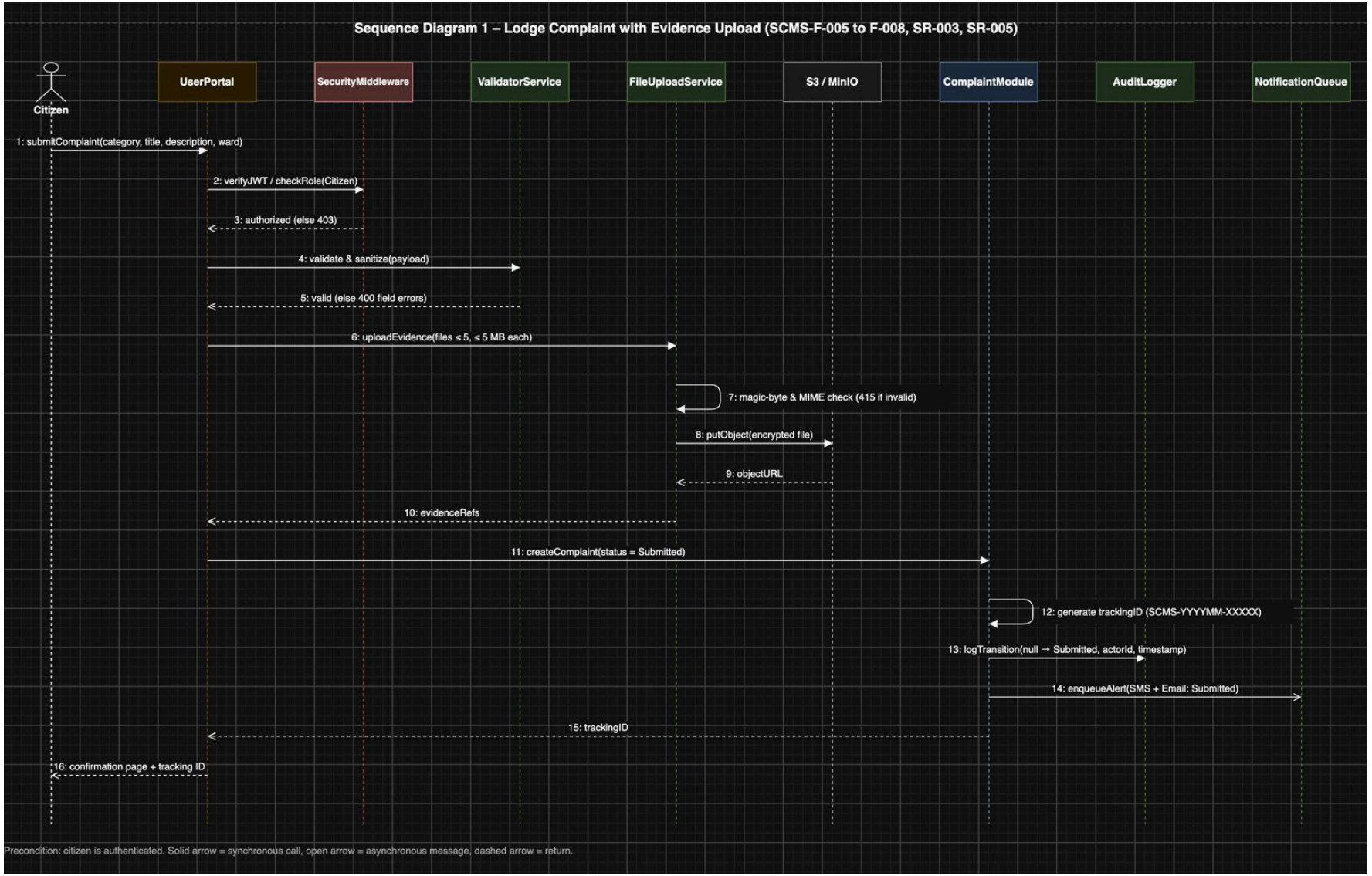


Figure 4.3 – Sequence Diagram 1: Lodge complaint with evidence upload

Sequence Diagram 2 covers the administrator flow for assigning a complaint (SCMS-F-011, F-013, F-014, F-017). After filtering and authorization, WorkflowModule commits the Submitted to Assigned transition in an ACID transaction, AuditLogger records it, and NotificationQueue dispatches SMS and email to the citizen and assigned officer within 30 seconds of the commit.

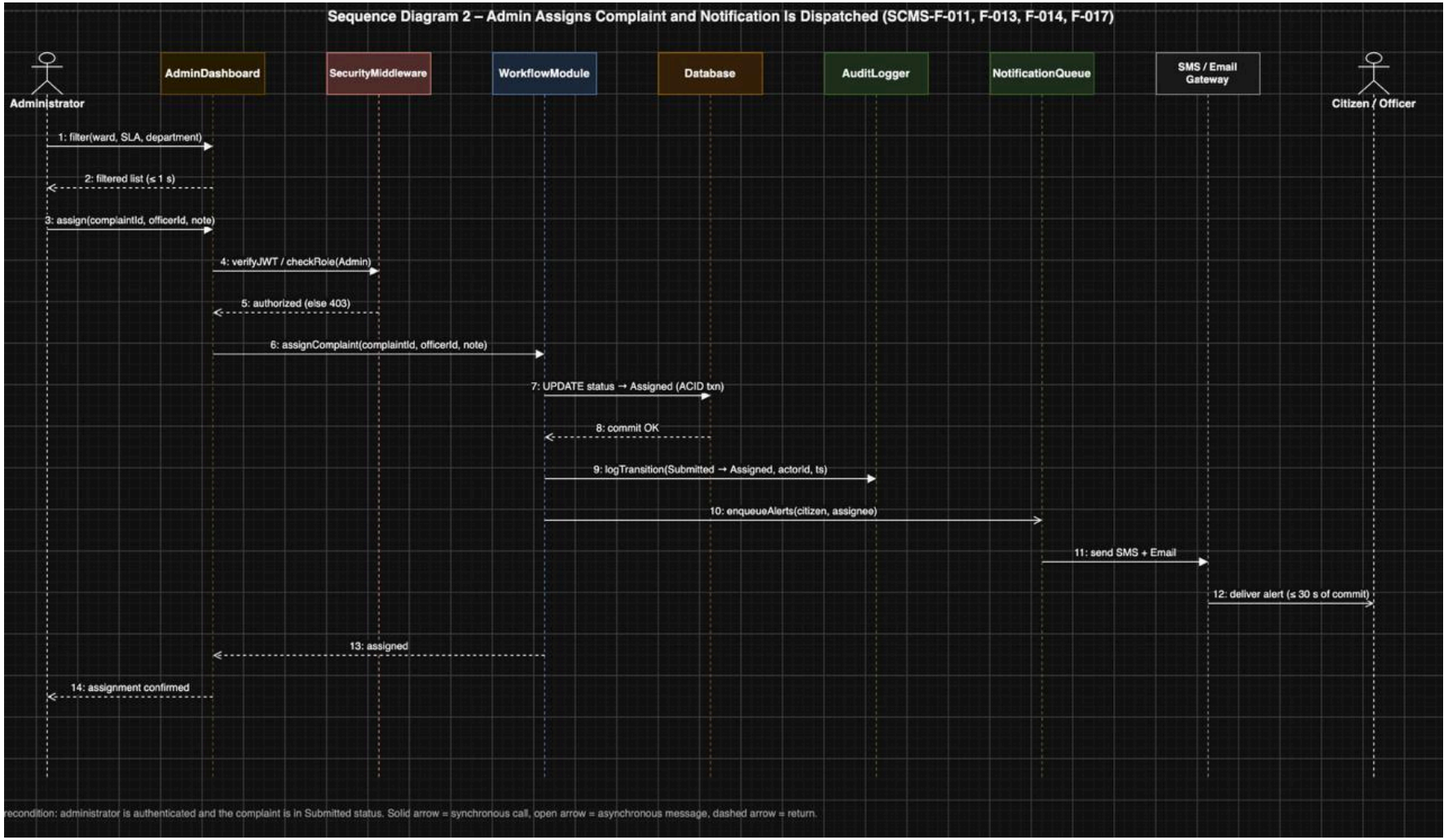


Figure 4.4 – Sequence Diagram 2: Admin assigns complaint and notification is dispatched

4.3 API Design

All endpoints are served over HTTPS (TLS 1.3), exchange UTF-8 application/json (multipart/form-data for uploads), and require a valid session cookie except where noted. The endpoints below are the proposed interface for the AuthModule, ComplaintModule with FileUploadService, TrackingModule, and WorkflowModule; exact paths may be adjusted during implementation.

4.3.1 Authentication: AuthModule

Item	Definition
Endpoint	/api/v1/auth/login
Method	POST (public)
Request	{identifier (email or phone), password}
Response 200	{status: "ok", role, expiresIn}; session token set in an HttpOnly, Secure, SameSite=Strict cookie
Errors	401 Invalid credentials (generic message); 423 Account locked for 30 minutes after 5 failed attempts; 400 Malformed request
Requirements	SCMS-F-002, F-004, SR-002, SR-004

4.3.2 Lodge Complaint: ComplaintModule and FileUploadService

Item	Definition
Endpoint	/api/v1/complaints
Method	POST (Citizen role, multipart/form-data)
Request	{category, title, description, ward, evidence[0..5]}: JPEG, PNG, or PDF, at most 5 MB each
Response 201	{trackingId (SCMS-YYYYMM-XXXXX), status: "Submitted", createdAt}
Errors	400 Missing or invalid fields; 401 Not authenticated; 403 Wrong role; 413 File larger than 5 MB or more than 5 files; 415 Magic-byte or MIME mismatch
Requirements	SCMS-F-005, F-006, F-007, F-008, SR-003, SR-005

4.3.3 Public Tracking: TrackingModule

Item	Definition
Endpoint	/api/v1/complaints/track/{trackingId}
Method	GET (public, no re-authentication)
Request	Path parameter trackingId
Response 200	{trackingId, status, lodgedAt, ward, department}; complainant PII masked
Errors	404 Unknown tracking ID
Requirements	SCMS-F-009

4.3.4 Assign Complaint: WorkflowModule

Item	Definition
Endpoint	/api/v1/admin/complaints/{id}/assign
Method	PATCH (Administrator role)

Item	Definition
Request	{officerId, note}
Response 200	{complaintId, status: "Assigned", assignedTo}; notification queued
Errors	401 Not authenticated; 403 Non-administrator (RBAC); 404 Unknown complaint or officer; 409 Invalid state transition
Requirements	SCMS-F-003, F-011, F-014, F-017

4.4 Error Handling, Logging & Monitoring

Error handling:

- Standardized JSON error body for all endpoints: {error: {code, message, traceId}}; field-level errors for validation failures so the UI can mark mandatory fields.
- Login failures return a generic message that does not reveal whether the account exists.
- A failed SMS/email gateway call never rolls back a committed complaint; the alert moves to a dead-letter retry queue.

Logging:

- No passwords, tokens, or unmasked PII in application logs.
- A separate immutable audit trail records status transitions with previous status, new status, actor ID, and timestamp, and records staff and administrator actions.

Monitoring:

- Availability: external uptime probes against health endpoints (target 99.5% per month).
- Performance: 90th-percentile response time (target 2.0 seconds or less) and filter latency (1 second).
- Notification delivery latency (30 seconds), retry/dead-letter queue depth, failed-login and lockout counts, and upload rejection rates.
- Backups: daily backup success and restore-time verification (under 30 minutes).

4.5 UX Design

- Citizen portal: mobile-first responsive layout, a dedicated 3-step complaint submission wizard, a persistent search bar for tracking numbers, and localized micro-copy.
- Staff dashboard: high-density workspace with sortable data tables, kanban lifecycle boards, interactive triage filters, and resolution confirmation modals.
- Accessibility: high-contrast palette conforming to WCAG 2.1 Level AA, full keyboard-only navigation, labelled ARIA attributes; Lighthouse accessibility score of at least 90.
- Performance on constrained connections: responsive behaviour over 3G/4G mobile networks.
- Client-side validation with red field banners for mandatory inputs, backed by server-side validation.

4.6 Open Issues & Next Steps

#	Open issue	Proposed next step
1	Backend framework not finalized (Django/Flask or Laravel) and session strategy not finalized (JWT cookies or Redis sessions).	Team decision recorded as an ADR before implementation.

#	Open issue	Proposed next step
2	SRS Sections 2.2 and 7.2 mention multi-factor authentication, but no functional requirement covers it.	Add an MFA requirement to the SRS or remove the mention; update AuthModule design accordingly.
3	The lifecycle includes Under Review and Appealed states, but no requirement describes the Appealed flow or the Under Review transition.	Add requirements and extend the workflow design and tests.
4	SRS Figure 7.2 omits reject with justification, export analytics, and configure categories/SLAs, which Section 7.2 text mentions.	Update the use-case diagram to match.
5	API paths and status codes in Section 4.3 are proposed, not yet reviewed against implementation.	Document in OpenAPI (Swagger) and review with the team.
6	No lifecycle state diagram, data model (ER diagram), or deployment diagram yet.	Add these diagrams in the next revision.
7	Request rate limiting on public endpoints is not specified in the SRS.	Consider adding it as a security requirement.

5. Appendices

5.1 Glossary

SCMS, SRS, STP, RTM, RBAC, SLA, JWT, MIME, ORM, ACID, ADR, STRIDE, TLS, PII, WCAG, OWASP: see Section 1.4. WCAG: Web Content Accessibility Guidelines. OWASP: Open Worldwide Application Security Project.

5.2 References

- SCMS Software Requirements Specification, Version 2.0, Team 8.
- SCMS Software Test Plan, Version 1.0, Team 8.
- IEEE 42010, Architecture description.
- OWASP Top 10.
- NIST SP 800-160, Systems Security Engineering.
- WCAG 2.1 Level AA guidelines.
- RFC 8446, TLS 1.3.

5.3 Tools

draw.io (UML component and sequence diagrams); Swagger/OpenAPI (recommended for API documentation).